

Given these problems, I will not discuss this approach to extending Python. If you're interested in it, then you could browse through the extension examples provided in the Python source directory.

The preferred way of extending Python is through extension modules created in the form of shared libraries. This way, you don't bloat Python itself with C/C++ chunks added to it, but load only the required extension module, on demand. Also, separate extension modules provide better modularity in the code of complicated or large applications. We will look at Ctypes, SWIG, Pyrex and Cython below; all of which allow you to create extensions in this manner.



Note: I used Puppy Linux 4.2.1 and Ubuntu 9.10 64-bit desktop edition to test the source code presented in the article, and to generate the screenshots.

Ctypes, a Python “insider”

Ctypes is a *foreign function interface* Python package, and part of the official Python distribution from version 2.5 onwards. Your Python code can use the Ctypes module to invoke C functions residing in shared libraries. Ctypes also does the data conversion between C and Python data types, and provides some general-purpose functions for working with shared libraries. You can create Python wrappers over your C shared libraries very easily, and code using Ctypes stands a better chance of being portable across all platforms on which Python runs.

To see Ctypes in action, type the following code in a Python file and run it:

```
from ctypes import CDLL

slibc = 'libc.so.6'
hlibc = CDLL(slibc)

iret = hlibc.abs(-7)
print iret
```

This Python code calls the GNU/Linux standard C library's *abs()* function. You can dynamically load any C shared library that contains functions following the *cdecl* calling convention (which is the default C calling convention on the x86 architecture), by passing its name to the *CDLL* class constructor. Alternately, use the *LoadLibrary()* method of the *CDLL* class. Ctypes provides other classes to load shared libraries that export functions under other calling conventions, like *stdcall* and *thiscall*. I encourage you to explore those on your own if you're interested—I'm focusing on the *cdecl* calling convention in this article. If you use the wrong Ctypes library-loading class for the calling convention used in functions exported by a shared library, you will encounter an exception. To learn more about calling conventions, follow the Wikipedia link in the *References* section.

Loading the shared library returns a handle to the library in the form of a *CDLL* instance. You can then invoke a function in the shared library as a method of the class instance. Simple, right? Try to call other standard C library functions like *sleep()*, *time()*, etc. You can also experiment with other *libc* functions that accept *None* or a single integer/long parameter. API documentation for *libc* is linked to in the *References* section.

The Ctypes utility function *find_library()* (in the *util* sub-module) finds the full name of any shared library given a library base name string (without the prefix “lib” and the extension and version number). The function returns *None* if it can't find a matching library. Run the following code:

```
from ctypes.util import find_library

llibs = ('bz2', 'c', 'm',)
```

```
for s in llibs:
    print (s + ': ' + str(find_library(s)))
```

The code finds the full names of the *bz2*, standard C and math libraries, respectively.

Though using Ctypes seems simple, there are enough ways to crash Python with it—in particular, passing unsupported Python data types in a Ctypes call to a library function. *None*, integers, longs, byte strings and Unicode strings are the only native Python data types/objects that you can directly pass as parameters in Ctypes function calls. (This was the reason we could pass an integer directly in the *abs()* call shown earlier.) *None* is passed as a C *NULL* pointer, byte strings and Unicode strings are passed as pointers to the memory block that contains their data; integers and longs are passed as the platform's default C *int* type.

To pass other data types into functions, you have to construct a proper data object for each variable that may not be directly passed. Ctypes' fundamental data type objects, which correspond to their counterparts in C, are: *c_char*, *c_wchar*, *c_byte*, *c_short*, *c_ushort*, *c_int*, *c_uint*, *c_long*, *c_ulong*, *c_float*, *c_double*, etc. Ctypes also has the basic pointer types, like *c_char_p*, *c_wchar_p* and *c_void_p*. You can instantiate these data objects, optionally passing an initialiser value of the corresponding Python data type to the constructor. For example, *hw=c_char_p("Hello, World")* and *us=c_ushort(-3)* creates the *hw* and *us* Ctypes data type instances.

Let's move on from calling functions in existing libraries to making a shared library of our own, and then using its exported functions in a small Python application.



Note: To keep article length down, some source code is not included in-line; you'll need to download the zip archive containing the files from www.linuxforu.com/article_source_code/extending-python.zip. Extract the contents of the archive in a folder you set aside for this experimentation.